深圳大学 SHENZHEN UNIVERSITY

课程 COURSE

Big Data Analytics
大数据分析 [2026 国际商务]



Lecture 7 — Supervised Learning Algorithms 第七讲·监督学习算法族
从最大间隔到树集成：函数类的四种写法

任课教师：戴谢尔 (Xieer Dai, David)

深圳大学 Shenzhen University · 第 3–14 周 · 星期四 11-12 节

上课地点：四方楼南 209

学习目标 Learning Objectives

完成本讲后，你应能够：

1. 写出最大间隔目标并解释支持向量 support vector 的作用；
2. 说明核技巧 kernel trick 为什么能隐式升维；
3. 用偏差-方差解释 k 的选择与维度灾难 curse of dimensionality；
4. 写出 CART 的基尼不纯度 Gini impurity 与信息增益 information gain；
5. 说清 bagging 与 boosting 的差别，并用 XGBoost / LightGBM / CatBoost 建表格基线；
6. 用 Optuna 调参、用 SHAP 解释，并说清 2026 的模型选型判据。

来源：对应课程大纲第 7 讲目标：掌握 SVM、KNN、CART、随机森林与梯度提升的原理、实现与调参。

本讲路线图 Roadmap（100 帧）



主线：框架 → 间隔（SVM）→ 近邻（KNN）→ 分段（CART）→ 组合（集成）

代码页 10 帧•样本页 14 帧；术语全程中英对照，代码逐行英文注释

与上一讲的衔接 From Linear Models to Algorithm Families

L6 留下的框架

三要素：函数类 function class、正则项 regularizer、优化算法 optimizer。L6 的四个模型都只是" 函数类 = 线性" 的特例。

L7 要做的事

换掉第一格：

- 间隔最大化 → SVM
- 局部平均 → KNN
- 分段常数 → CART
- 组合多个分段常数 → 随机森林 / GBDT

同一个损失、同一套调参纪律、同一套评估协议——变的只是" 假说空间长什么样"。这是本讲最想让你记住的一句话。

来源：框架承 L6 帧 90；四种函数类的划分据 ESL 与 ISLR。

本讲输出物 Deliverables

1) 算法网格

七个模型在「参数量 × 可解释性」二维网格上的定位(帧 18)

2) 可运行代码

10 个程序：SVM/KNN/CART/AdaBoost/RF/GBDT/Optuna/SHAP/代价敏感，逐行英文注释

3) 术语表

72 条中英对照（CSV + Word/Excel + 合并版）

所有数字都来自可复现的程序：`website/code/lecture07/`，种子固定 20260708，一条 `run_all.py` 重建 15 张图与 10 张表。

来源：程序与样本随课程仓库发布；术语表另附可打印版本与 L6+L7 合并版。

全讲导览 Map of the Lecture

部分	帧	关键问题
共同框架	7–20	七个算法共享什么？谁决定表达能力？怎么调参？
支持向量机	21–40	哪条分界线最好？核是什么？C 与 γ 怎么选？
K 近邻	41–54	邻居怎么投票？k 怎么选？维度为什么是敌人？
决策树	55–68	怎么分裂？怎么剪枝？为什么单树不够？
集成与 GBDT	69–86	袋装与提升差在哪？为什么表格数据首选 GBDT？
调参与解释	87–96	怎么搜参数？怎么解释？2026 怎么选型？
工具与术语	97–100	用什么库？术语怎么背？

读法：每个算法都走一遍直觉 → 形式化 → 偏差-方差 → 样本图 → 代码 → 调参与坑， 到帧 18 你会发现它们其实是同一张表的不同格子。

来源：导览为本讲安排；各部分均在后续帧展开。

为什么先讲框架 Why a Common Frame First

七个算法的”长相”

SVM 讲间隔、KNN 讲距离、树讲分裂、森林讲抽样、提升讲残差——名词各不相同，容易学成七套互不相干的知识。

其实只有三层

所有监督学习算法都在回答三个问题：

- 候选函数长什么样？（函数类）
- 怎么防止它太复杂？（正则项）
- 怎么找到最好的那个？（优化）

本部分的目的：先给你一个可以往里装东西的框架，后面每讲一个算法，你只需要问” 它改的是哪一格”。

来源：框架据课程讲义” 监督学习者的结构” 一页；表述为本讲组织。

监督学习者的三要素 Three Ingredients



函数类

线性、树、近邻、核、神经网络。决定能表达什么。

正则项

L_1/L_2、剪枝、最大深度、早停、C。决定有多保守。

优化算法

闭式解、坐标下降、SMO、贪心分裂、函数空间梯度。决定多快找到解。

这三格填满，模型就确定了。所以" 比较两个算法" 永远可以拆成 比较函数类、比较正则、比较优化三件事。

来源：三要素结构与课程讲义一致；三格的解释为本讲补充。

函数类决定”能表达什么” What the Function Class Buys

函数类	形式	能表达 / 不能表达
线性	$w^\top x + b$	加性、单调；不能表达交互与阈值
分段常数（树）	叶节点常数的阶梯函数	阈值与交互；边界是轴平行的
局部平均（KNN）	邻域内取平均	任意形状；需要大量样本覆盖空间
核方法（SVM）	核展开 $\sum_i \alpha_i K(x_i, x)$	光滑的非线性；核决定光滑度

关键区分：树给的是轴平行的阶梯，核给的是光滑的曲线，KNN 给的是局部的拼贴。表达能力的差别就是这三个词。

来源：函数类对比据 ESL 与 ISLR；” 轴平行/光滑/局部” 的提法为本讲归纳。

正则项决定”有多保守” What the Regulariser Buys

同一个想法的不同名字

- 线性模型：$\lambda\|w\|_2^2$（Ridge）；
- SVM：惩罚参数 penalty parameter C（C 小 $=$ 正则强）；
- 树：最大深度、最小叶样本、代价复杂度剪枝 cost-complexity pruning α；
- 森林/提升：树的数量、学习率、子采样比例；
- 神经网络：dropout、早停 early stopping。

一句话

都是复杂度预算：花多少参数去拟合训练数据。预算越紧，方差越小、偏差越大。

注意 C 的方向：C 越大正则越弱（越不允许违反间隔）；λ 越大正则越强。两个名字方向相反，考试常考。

来源：各算法的正则写法据 ESL/ISLR 与各库文档；方向提醒为本讲补充。

优化算法决定”怎么找到解” Which Optimiser

模型	优化方法	特点
线性/逻辑回归	闭式解、牛顿法、梯度下降	凸、可检验
SVM	序列最小优化 sequential minimal optimisation (SMO)	凸二次规划的对偶
树	贪心分裂 greedy splitting（精确或直方图）	非凸、每步局部最优
森林	并行训练 + 平均	无优化，只有抽样
提升	函数空间梯度下降 gradient descent in function space	串行、每轮拟合残差

优化的性质决定稳定性：凸问题结果唯一（SVM、线性模型），贪心分裂结果依赖随机种子（树、森林、提升）。所以树模型必须报告种子。

来源：优化方法据各算法原始文献与库实现；稳定性提醒为本讲补充。

三句话：灵活、限制、学会限制多少

课程讲义的版本

1. 用灵活的函数形式（flexible functional forms）把模式装进去；
2. 限制表达力（limit expressiveness）防止拟合噪声；
3. 学会该限制多少（learn how much to regularise）——这就是调参。

第 3 句的工具

留出样本 hold-out sample：切一块不参与训练的数据；交叉验证 cross-validation：把留出重复若干次。

为什么必须”学”

正则强度不能由训练误差决定（训练误差总是越松越小），必须由样本外误差决定。

来源：三句话据课程讲义；留出与交叉验证的表述见 L5。

防火墙原则 The Firewall Principle

三个集合

- 训练集 training set：拟合参数；
- 验证集 validation set：选超参数、选模型；
- 测试集 test set：只在最后用一次，报告最终成绩。

防火墙的含义

信息只能单向流动：验证集可以影响选哪个模型，测试集不能影响任何决策——否则最终成绩就是乐观偏差。

项目里最常见的违规：用测试集挑阈值、挑阈值、挑特征。一旦发生，报告里的样本外指标就不能用了（衔接 L5 的泄漏清单）。

来源：防火墙原则据课程讲义；违规清单衔接 L5 的数据泄漏部分。

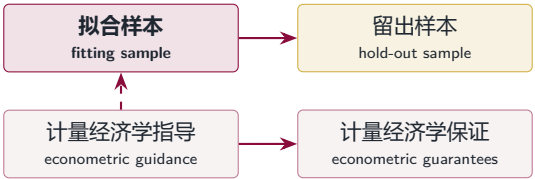
调参的三种做法 Three Ways to Tune

做法	做法	代价
留出法	切一块验证集，试着调	便宜；方差大，浪费数据
K **折交叉验证**	轮留一折，取平均	稳；K 倍训练成本
嵌套交叉验证	外层评估、内层调参	最诚实；成本最高

本讲的代码页统一用4–5 折交叉验证做调参、独立测试集报成绩。这也是项目报告里应该写清楚的” 调参协议”。

来源：三种做法据 ISLR 与 scikit-learn 文档；统一协议为本讲约定。

一个机器学习练习的结构 Structure of an ML Exercise



两层含义

上面一行是机器学习：用拟合样本训练、用留出样本评估；下面一行是计量经济学：变量选择与设定需要理论指导，估计量需要可解释的保证。

实践建议

先用经济逻辑定"该进哪些变量、方向应该是什么"，再用交叉验证定"怎么拟合"。两者都写进报告。

来源：结构图据课程讲义（Athey 风格）；实践建议为本讲补充。

研究者的四个选择 Four Researcher Choices

1) 损失	2) 数据管理	3) 特征表示	4) 函数类与正则
平方、绝对值、交叉熵、合页……它决定了"错"的定义。	怎么切分、怎么处理时间顺序、怎么防泄漏。	原始变量、交互项、标准化、嵌入。	线性/树/森林/近邻；λ、深度、C。

四件事里，前三件比第四件更重要——这是 2026 的共识：数据和特征决定上限，模型只决定离上限多近。课程讲义把这四条列为"重要的研究者选择"。

来源：四个选择据课程讲义；"前三件更重要"的排序为本讲补充（与 L5 的特征工程讨论一致）。

从 LASSO 到神经网络：函数类与正则项

函数类	正则项（怎么限制表达力）	本讲的对应帧
线性	LASSO、Ridge、Elastic Net	L6 帧 76–81
决策/回归树	深度、叶节点数、最小叶样本、信息增益阈值	帧 55–68
随机森林	树的个数、每棵树的变量数、样本量、复杂度	帧 69–75
最近邻	邻居个数 k	帧 41–54
核回归	带宽 bandwidth	帧 28–34
样条	结点数、阶数	（不展开）
神经网络	层数、宽度、连接、dropout、早停	帧 93、L9

这张表来自课程讲义，是同一件事的七种写法：每一行左边是" 能多灵活"，右边是" 怎么限制它"。

来源：对照表据课程讲义"From LASSO to neural nets" 一页；帧号为本讲对应位置。

算法网格：七个模型在哪里 The Algorithm Grid

模型	参数量	可解释性	典型样本量
线性回归	少	高	小
逻辑回归	少	高	小
SVM	中	低	中小 ($< 10^5$)
KNN	少	中	小 (要覆盖空间)
单树	中	高	小
随机森林	多	中	中
GBDT	多	中 (+SHAP)	中大

两条经验

- 解释优先、样本小 $\rightarrow$ 线性 + 正则；
- 精度优先、表格数据 $\rightarrow$ GBDT；
- 需要" 为什么"$\rightarrow$ 再加 SHAP（帧 91）。

记住这张网格，后面每个算法都往里填一格；帧 93 会用它解释 2026 的选型判据。

来源：网格为本讲整理；样本量区间为经验值，具体取决于特征维度。

统一的评测口径 One Scoring Protocol

三条纪律

1. 同一份数据、同一个切分（固定种子）；
2. 同一组指标（回归 R^2/RMSE，分类 AUC/PR/校准）；
3. 报告均值与波动（交叉验证的标准差），不只报一个数。

为什么重要

不同论文/不同模型的结果常常不可比，因为切分与指标不同。你自己报告时把这三条写清楚，结论才站得住。

本讲所有对照实验都遵守这三条：种子 42、同一个 `train_test_split`、同一组指标。代码页里你会反复看到同样的两行。

来源：评测纪律据 L5 的评估协议；本讲沿用同一套。

小结：框架记住了，剩下只是换格子

三要素

函数类、正则项、优化算法——七个算法都是这三格的组合。

三句话

灵活的函形式 + 限制表达力 + 学会限制多少。

接下来四个格子

- 间隔：SVM（帧 21–40）
- 局部：KNN（帧 41–54）
- 分段：CART（帧 55–68）
- 组合：森林与提升（帧 69–86）



来源：小结为本讲整理。

直观：哪条分界线最好？Which Line Is Best?

问题

两类点分开时，能画的分界线有无穷多条。它们在训练集上都 100% 正确，但在新数据上表现不同。

直觉答案

选离两类都最远的那条——留出最大余量。这条思路叫最大间隔 maximum margin，是 SVM 的全部出发点。

为什么" 最远" 更稳？因为样本有噪声：测试点会偏离训练分布一点点，边界两侧留的余量越大，偏离后仍落在正确一侧的概率越高。

来源：最大间隔的直觉据 ESL 与 ISLR；噪声解释为本讲补充。

超平面、法向量与距离 Hyperplane and Distance

超平面的写法

$$w^\top x + b = 0,$$

其中 w 是法向量 normal vector，b 是偏移。Hesse 法式要求 $\|w\| = 1$，SVM 里不要求（便于优化）。

点到超平面的距离

$$d(x) = \frac{|w^\top x + b|}{\|w\|}.$$

两个要点：(1) w 的方向决定边界的朝向，b 只决定平移；(2) 距离公式里的 $\|w\|$ 是分母——所以"让间隔最大"就等价于"让 $\|w\|$ 最小"。

来源：超平面与距离公式据 ESL 第 12 章；Hesse 法式的说明据课程讲义。

最大间隔的优化问题 The Maximum-Margin Problem

形式化

标签取 $y_i \in \{-1, +1\}$，要求

$$y_i\big(w^\top x_i + b\big) \geq 1 \quad \forall i,$$

此时两类到边界的最小距离都是 $1/\|w\|$，间隔 margin 宽为 $2/\|w\|$。于是

$$\min_{w,b} \ \frac{1}{2}\|w\|^2 \quad \text{s.t.} \quad y_i\big(w^\top x_i + b\big) \geq 1.$$

为什么右边是 1？可以随便取——因为 w, b 同乘一个常数时超平面不变，把最近点的函数间隔规范化成 1 不损失一般性。

A convex quadratic programme with linear constraints.

来源：推导据 ESL 第 12 章与课程讲义；规范化说明为本讲补充。

【代码】SVM：硬间隔、软间隔与惩罚参数 C

```python
from sklearn.svm import SVC, LinearSVC                # the two SVM interfaces
Cs = [0.01, 0.1, 10.0]                                # weak, moderate and strong penalty
for C in Cs:                                          # loop over the penalty values
    svm = SVC(kernel="linear", C=C)                   # linear SVM with this C
    svm.fit(X_train, y_train)                         # fit on the training split
    n_sv = len(svm.support_)                          # number of support vectors
    w_norm = float(np.linalg.norm(svm.coef_[0]))      # norm of the normal vector
    margin_width = 2.0 / w_norm                       # width of the margin band
    acc = accuracy_score(y_test, svm.predict(X_test))  # test accuracy
    rows.append({"C": C, "support_vectors": n_sv,     # penalty and support vector count
                 "w_norm": round(w_norm, 3),          # normal vector length
                 "margin_width": round(margin_width, 3), # margin width in feature units
                 "test_acc": round(acc, 4)})          # out-of-sample accuracy
sv = models[C].support_vectors_                       # support vectors of the fit
ax.scatter(sv[:, 0], sv[:, 1], s=70, facecolors="none", edgecolors=0KABE[2], # circle the support vectors
           linewidths=1.2, label="support vectors")   # legend label
```

实测（本讲合成样本）：$C = 0.01 \to |w| = 0.54$、间隔 3.71、测试 0.683；$C = 0.1 \to$ 间隔 1.99、0.800；$C = 10 \to$ 间隔 1.56、0.800。**间隔越宽越保守**。完整程序 `17_01_svm_margin.py`。

拉格朗日对偶：为什么会出现”内积”

对偶形式（结论）

$$\max_{\alpha} \sum_i \alpha_i - \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j x_i^\top x_j$$

s.t. $\alpha_i \geq 0$, $\sum_i \alpha_i y_i = 0$。解出 α 后 $w = \sum_i \alpha_i y_i x_i$。

两个关键观察

1. 数据只以内积 inner product $x_i^\top x_j$ 出现；
2. 多数 $\alpha_i = 0$ ——只有少数样本起作用（稀疏性 sparsity）。

第 1 条是核技巧的入口：把内积换成核函数，就等于在另一个空间里做内积。第 2 条说明为什么 SVM 的模型只由少数样本决定。

来源：对偶形式据 ESL 第 12 章；两点观察为本讲强调。

支持向量：只有边界附近的点算数 Support Vectors

定义

满足 $y_i(w^\top x_i + b) = 1$ 的样本叫支持向量support vector——它们恰好落在间隔边界上，是决定 w 的唯一样本。

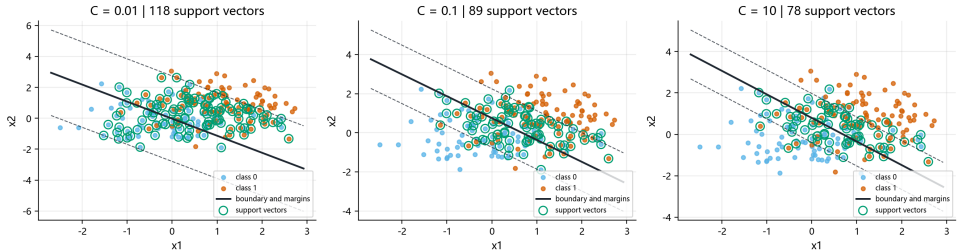
含义

- 移动远离边界的点，边界不变；
- 所以 SVM 对远离边界的噪声不敏感；
- 但边界附近的点极其重要（也极其脆弱）。

实测：本讲样本在 $C = 0.01$ 时有 118 个支持向量，$C = 10$ 时降到 78 个——C 越大越"挑剔"，只留最难的样本。

来源：支持向量的定义据 ESL；实测数字来自程序 l7_01。

【样本 + 可视化】软间隔与 C：容错多少 Soft Margin



同一份样本、三个 C：$C = 0.01$ 时间隔宽（3.71）、支持向量多（118）、测试 0.683；$C = 0.1$ 时间隔 1.99、测试 0.800；$C = 10$ 时间隔 1.56、支持向量 78。虚线是间隔边界，圆圈是支持向量。

程序 `l7_01_svm_margin.py` ·样本 `l7_two_class.csv` ·汇总 `l7_svm_margin.csv`

核技巧：不显式升维 The Kernel Trick

想法

如果原空间分不开，就映射到高维 $\phi(x)$ 再分。但对偶形式只需要 $\phi(x_i)^\top \phi(x_j)$，所以我们只需定义

$$K(x, z) = \phi(x)^\top \phi(z),$$

不用真的算出 $\phi(x)$。

为什么厉害

- 计算量只取决于 K，与维度无关；
- 高维（甚至无穷维）空间也能用；
- 代价：要选核、选核参数（帧 31）。

Mercer 条件保证：只要 K 是半正定的，就一定存在某个 ϕ 与之对应。所以" 随便写个函数当核" 不行，必须满足条件。

来源：核技巧据 ESL 第 12 章与课程讲义；Mercer 条件为本讲补充。

四种常用核 Four Standard Kernels

核	形式	特点与适用
线性 linear	$x^\top z$	高维稀疏（文本）；等价于线性 SVM
多项式（同质）	$(x^\top z)^d$	能表达 d 阶交互；需定 d
多项式（非齐次）	$(\gamma x^\top z + r)^d$	更灵活；$r = 0$ 退化为同质
径向基 radial basis function (RBF)	$e^{-\gamma \lVert x - z \rVert^2}$	最常用；γ 控制" 局部性"
sigmoid	$\tanh(\gamma x^\top z + r)$	类似两层网络；不总是半正定，慎用

RBF 的直观：两个点的相似度随距离指数衰减，γ 决定衰减多快——γ 大则" 只认很近的点"，边界会贴合单点（过拟合）。

来源：四种核据课程讲义与 scikit-learn 文档；sigmoid 核的风险为本讲补充。

【代码】核选择与 γ、C 网格 Kernel and Grid Search

```python
kernels = {"linear": {"C": 1.0}, "poly": {"C": 1.0, "degree": 3}, # linear and cubic polynomial
         "rbf": {"C": 1.0, "gamma": 1.0}, "sigmoid": {"C": 1.0, "gamma": 0.5}} # RBF and sigmoid
for name, params in kernels.items():                  # loop over the four kernels
    model = SVC(kernel=name, **params)                # build the SVM
    model.fit(X, y)                                   # fit on the whole sample
    scores = cross_val_score(model, X, y, cv=5, scoring="accuracy") # five-fold accuracy
    rows.append({"kernel": name, "cv_mean": round(float(scores.mean()), 4), # mean accuracy
                 "cv_std": round(float(scores.std()), 4), # spread across the folds
                 "support_vectors": len(model.support_)}) # support vector count
param_grid = {"C": [0.1, 1.0, 10.0, 100.0], "gamma": [0.05, 0.2, 1.0, 5.0]} # a small grid
search = GridSearchCV(SVC(kernel="rbf"), param_grid, cv=5, scoring="accuracy") # five-fold search
search.fit(X, y)                                      # run the search
print("best RBF: C=%.2f gamma=%.2f | cv=%.4f" % (     # report the winner
    search.best_params_["C"], search.best_params_["gamma"], search.best_score_)) # the three numbers
```

实测：四种核的交叉验证准确率——线性略低、RBF 最优；网格搜索选出 $C = 10$、$\gamma = 0.20$，交叉验证 0.923。完整程序 `17_02_svm_kernels.py`。

γ 的作用：局部到什么程度 What Gamma Controls

γ 小

RBF 退化为近似线性（相似度衰减慢），边界平滑，偏差大、方差小——类似欠拟合。

γ 大

只认很近的点，边界贴合每个样本，方差大、偏差小——类似过拟合。

与 C 的配合

C 控制允许多少违反，γ 控制相似度的尺度。两者一起决定边界的" 精细程度"，所以必须联合网格搜索，不能各调各的。

经验起点：先标准化特征，再在 $\gamma \in [1/(p \cdot \mathrm{var}), 1/p]$ 附近搜，C 用 10^{-2} 到 10^{3} 的对数网格。

来源：γ 的解读据 scikit-learn 文档；起点建议为本讲经验值。

SVM 的优缺点（scikit-learn 口径）Pros and Cons

优点

- 高维空间有效；
- 维度高于样本数时仍可用；
- 只用支持向量，内存友好；
- 核函数灵活（可自定义）。

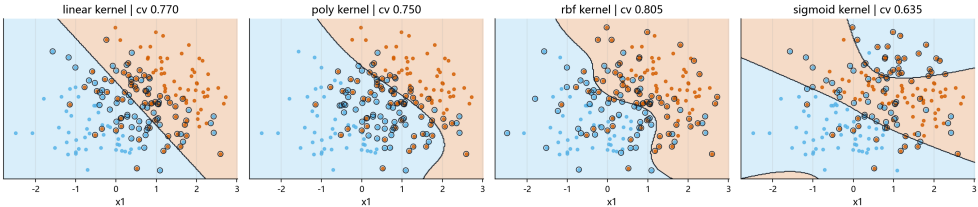
缺点

- 特征数远大于样本数时，核与正则的选择极易过拟合；
- 不直接给概率，需昂贵的 5 折交叉验证 (Platt)；
- 训练复杂度 $O(n^2)$–$O(n^3)$，大规模数据不友好。

课程讲义原文："SVMs do not directly provide probability estimates, these are calculated using an expensive five-fold cross-validation."——概率要不要，是选 SVM 前必须回答的问题。

来源：优缺点据 scikit-learn 用户指南（课程讲义引用同一页）；复杂度说明为本讲补充。

【样本 + 可视化】四种核的决策边界 Four Kernels



同一份非线性样本，四种核：线性只能切一刀（准确率最低）；三次多项式边界有弯曲但受 d 限制；RBF 贴合数据形状（调参后 0.923）；sigmoid 在本样本上几乎失效（接近随机）。空心点是支持向量。

程序 `l7_02_svm_kernels.py` ·汇总 `l7_svm_kernels.csv`

SVM 的概率输出与校准 Probability from a Margin

Platt 缩放

用逻辑回归拟合 $P(y=1 \mid f)=\sigma(Af+B)$，其中 f 是决策函数值。A, B 用内部交叉验证估计——所以 scikit-learn 里 `probability=True` 会明显变慢。

实践建议

- 只要硬分类 $\rightarrow$ 不开概率；
- 要算代价、要报概率 $\rightarrow$ 开概率并检查校准曲线（L6 帧 73）；
- 大规模数据 $\rightarrow$ 改用线性 SVM 或 GBDT。

术语衔接：概率校准 probability calibration 在 L6 用等渗回归演示过；SVM 的 Platt 缩放是它的经典版本。

来源：Platt 缩放据 Platt (1999) 与 scikit-learn 文档；建议为本讲补充。

SVM 的规模边界 Scaling Limit

情形	推荐做法	原因
$n < 10^4$，p 中等	核 SVM + 网格搜索	精度高、实现简单
n 大（$> 10^5$）	线性 SVM（`LinearSVC`）或 SGD	核矩阵内存爆炸
$p \gg n$（文本）	线性核 + 强正则	高维稀疏下线性往往够用
需要概率与解释	逻辑回归 / GBDT	原生概率、可算重要性

一句话：SVM 是" 中等规模、中等维度" 的利器；大规模表格数据请交给帧 81 的梯度提升。

来源：规模建议据 scikit-learn 文档与实践经验。

什么时候选 SVM When to Choose SVM

选它

- 样本中等（几千到几万）、特征数中等；
- 边界是非线性但光滑的；
- 需要" 最大间隔" 这种稳健性论证。

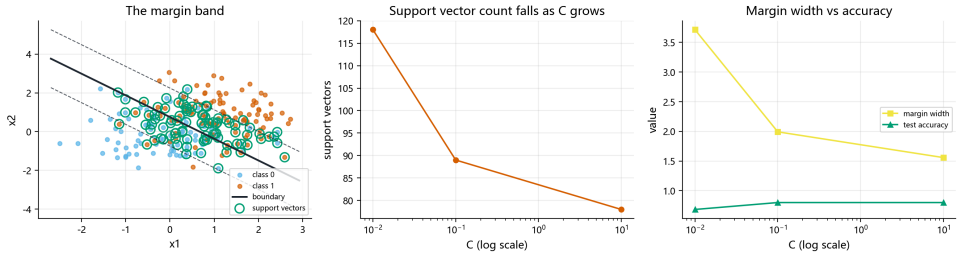
不选它

- 需要概率与阈值定价（用逻辑回归/GBDT）；
- 特征异构（类别多、缺失多）——树更省事；
- 需要解释" 每个变量怎么影响结果"——用线性模型 + SHAP。

判据可以总结成一句：要" 几何上的稳健" 选 SVM，要" 业务上的可解释" 选别的。

来源：选型判据为本讲整理，结合 L6 的可解释性讨论。

【样本 + 可视化】支持向量与间隔带 Support Vectors



左：一条边界、两条虚线（间隔边界）、圆圈标出的支持向量；中：C 增大时支持向量从 118 个降到 78 个；右：间隔宽度随 C 单调变窄，而测试准确率在 $C = 0.1$ 附近达到平台——间隔不是越宽越好，要看样本外。

程序 `17_01_svm_margin.py` ·图与帧 27 同源，便于前后对照

SVM 在业务里怎么读 Reading an SVM in a Report

能说的

- 判别能力（AUC、准确率、间隔宽度）；
- 哪些样本是" 边界案例"（支持向量）——可用于人工复核；
- 对特征缩放敏感 → 说明预处理做了什么。

不能说的

- " 系数每增加 1 单位，概率上升多少"——SVM 没有这个；
- 变量的重要性排序——需要另做置换重要性或 SHAP。

项目报告里如果用 SVM，别写" 系数解释"；改用" 边界案例清单 + SHAP 摘要" 来支撑业务结论（帧 91）。

来源：业务读法为本讲整理；与项目评分表" 结果与验证" 一栏对应。

陷阱：不标准化就让 RBF 失效 Standardise First

问题

RBF 的相似度基于欧氏距离。如果一列以" 元" 计、另一列以" 年" 计，距离几乎完全由大尺度那列决定——核实际上只看了一个特征。

做法

用 `StandardScaler` 或 `MinMaxScaler`，且只在训练集上拟合缩放参数（与 L6 帧 67 的坑同源）。

检验方法：标准化前后各跑一次交叉验证，看分数差多少。差得多就说明尺度问题严重。

来源：标准化对核方法的影响据 scikit-learn 文档与实践；检验方法为本讲补充。

小结：SVM 是”几何派”

一句话

在无数条分界线里，选离两边都最远的那条；分不开就升维，而升维靠核。

两个旋钮

C（允许多少违反）与 γ（相似度多局部），必须联合搜索。



下一部分换一种完全不同的思路：不找” 最好的线”，而是看邻居。

来源：小结为本讲整理。

直觉：看邻居投票 k-Nearest Neighbours

分类

找离新样本最近的 k 个训练样本，多数票决定类别；$k = 1$ 时直接取最近的那个样本的类别。

回归

取 k 个邻居的平均值。

这是最" 朴素" 的机器学习：不训练，只存储——所有计算发生在预测时。所以它又叫懒惰学习 lazy learning（相对于" 急切学习"）。

来源：k-NN 的定义据课程讲义与 ISLR 第 2 章；懒惰学习的提法为本讲补充。

历史与定位 History and Position

来历

Fix & Hodges (1951) 提出最近邻判别，Cover 与 Hart (1967) 给出理论性质。它是非参数方法的典型代表。

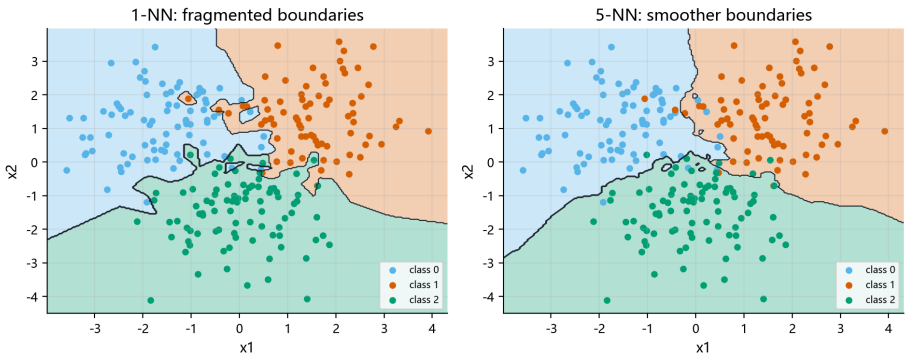
非参数的含义

不假设"$y = f(x)$ 一定属于某个有限族"；参数个数随训练样本增长。所以它没有"模型参数"，只有超参数 k。

理论上的经典结论：$k = 1$ 的错误率不超过贝叶斯最优错误率的两倍；当 $n \to \infty$、$k \to \infty$、$k/n \to 0$ 时，k-NN 一致收敛到贝叶斯最优。

来源：历史与理论据 Cover & Hart (1967) 与课程讲义；结论为本讲概述。

【样本 + 可视化】1-NN 与 5-NN 的分类地图



同一份三类样本：1-NN 的边界碎裂，每一点都划出一个小区域（高方差）；5-NN 的边界平滑得多。留出集准确率：$k = 1$ 为 0.833，$k = 5$ 为 0.931——平滑在这里换来了泛化。

程序 `17_03_knn_k_curve.py` ·样本 `17_knn_sample.csv`

统计设定与距离度量 Statistical Setting

设定

给定训练对 $\{(x_i, y_i)\}$ 与某种范数 $\|\cdot\|$，对新样本 x 把训练样本按距离排序：

$$\|x_{(1)} - x\| \le \|x_{(2)} - x\| \le \cdots$$

取前 k 个投票/平均。

距离的选择

- 欧氏（L_2）：默认；
- 曼哈顿（L_1）：对离群点更稳；
- 余弦：文本与稀疏向量；
- 马氏：考虑特征相关与尺度。

距离的选择等价于对" 相似" 的定义。在项目里，选余弦还是欧氏应该由业务含义决定，而不是默认欧氏。

来源：设定据 ESL 第 2 章与课程讲义；距离选择建议为本讲补充。

距离度量的影响 How the Metric Changes the Answer

同一个点，不同的邻居

在标准化后的二维数据上，欧氏距离会让尺度大的特征主导邻居的选择；余弦只看方向，忽略长度。

实践中的三条

1. 先标准化（否则距离被量纲绑架）；
2. 文本用余弦 + TF-IDF 权重；
3. 特征多且相关时考虑马氏距离或先降维。

代码页（帧 47）会用同一份数据对比欧氏与曼哈顿的 k 曲线，你会发现最优 k 与最优准确率都可能不同。

来源：度量影响据实践与 ISLR；三条做法为本讲整理。

k 的选择 Choosing k

k 小

边界贴合训练数据，偏差小、方差大；$k = 1$ 时训练误差为 0 但极易受噪声影响。

k 大

平均掉了噪声，方差小、偏差大；$k = n$ 时永远预测多数类。

课程讲义的原话

"Generally, larger values of k reduce the effect of the noise on the classification, but make boundaries between classes less distinct."——所以 k 要用交叉验证选。

经验起点：$k \approx \sqrt{n}$，然后在这个值附近搜。本讲样本（$n = 240$）的最优 $k = 3$，交叉验证 0.921。

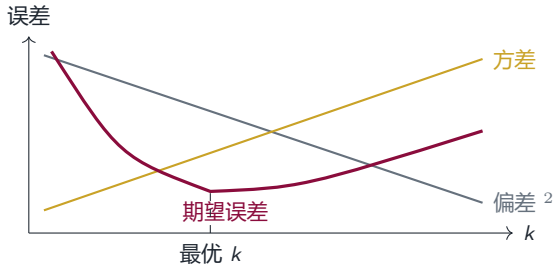
来源：原话据课程讲义；起点建议与实测结果来自程序 l7_03。

【代码】KNN：k 曲线与距离度量

```python
from sklearn.neighbors import KNeighborsClassifier     # the k-NN classifier
ks = list(range(1, 41))                                # candidate neighbourhood sizes
for k in ks:                                           # loop over the neighbourhood sizes
    model = KNeighborsClassifier(n_neighbors=k)        # the classifier for this k
    scores = cross_val_score(model, X, y, cv=5, scoring="accuracy") # five-fold accuracy
    model.fit(X, y)                                    # fit on everything for the error curve
    train_acc = accuracy_score(y, model.predict(X))    # in-sample accuracy
    cv_rows.append({"k": k, "cv_mean": round(float(scores.mean()), 4), # cross-validated accuracy
                    "cv_std": round(float(scores.std()), 4), # fold spread
                    "train_acc": round(float(train_acc), 4)}) # in-sample accuracy
best_k = int(cv_table.loc[cv_table["cv_mean"].idxmax(), "k"]) # best k by cross-validation
for k in (1, 5, best_k):                               # the three values we discuss
    model = KNeighborsClassifier(n_neighbors=k).fit(X_train, y_train) # fit on the training split
    holdout[k] = round(float(accuracy_score(y_test, model.predict(X_test))), 4) # test accuracy
```

实测：$k = 1$ 交叉验证 0.863、留出 0.833；$k = 3$ 交叉验证 0.921；$k = 5$ 留出 0.931；$k \geq 20$ 后曲线走平。完整程序 `17_03_knn_k_curve.py`。

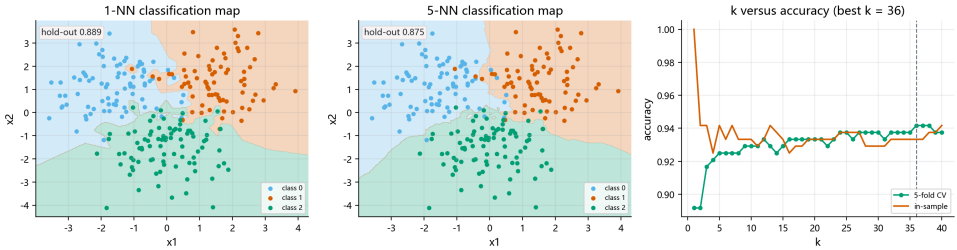
k 的偏差-方差视角 A Bias-Variance View of k



k 是最纯粹的复杂度旋钮：它不改变函数形式，只改变" 平均多少个邻居"。所以 k-NN 是讲偏差-方差最好的教具——一条曲线就能讲完。

来源：偏差-方差与 k 的关系据 ESL 第 2、7 章；本图自绘。

【样本 + 可视化】k 与准确率：一条曲线讲完



左、中：$k = 1$ 与 $k = 5$ 的分类地图（同帧 43）；右：k 从 1 到 40 的曲线——训练准确率单调下降（$k = 1$ 时 1.000），交叉验证先升后降，峰值在 $k = 3$（0.921）。两者之间的缺口就是过拟合的度量。

程序 `17_03_knn_k_curve.py` ·曲线表 `17_knn_k_curve.csv`

维度灾难 The Curse of Dimensionality

现象

维度升高时，最近邻与最远邻的距离趋于相同——" 近邻" 不再有区分度。

后果

k-NN 需要样本覆盖空间；p 维空间中体积随 p 指数增长，固定 n 时样本变得极其稀疏。

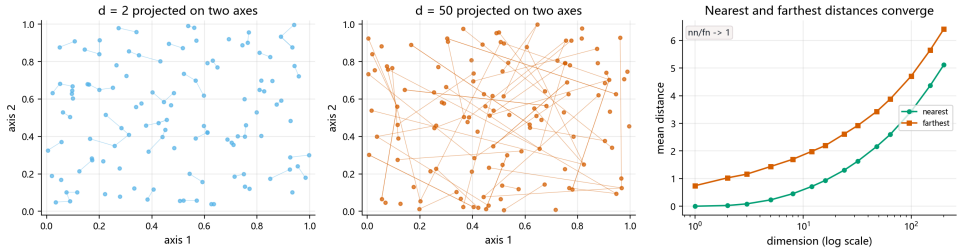
三条对策

1. 降维（PCA，L5 帧 64）；
2. 特征选择（去掉无关变量）；
3. 换对维度不敏感的模型（线性、树）。

本讲实测：单位超立方体内 300 个点，最近邻/最远邻距离比从 $d = 1$ 的 0.002 升到 $d = 200$ 的 0.799——几乎" 所有点一样远"。

来源：现象据 Beyer et al. (1999) 与 ESL；实测数字来自程序 l7_04。

【样本 + 可视化】维度升高后距离集中



左、中：二维投影下，每个点连向它的最近邻——二维时邻居" 真的在附近"，五十维时连线看起来很随机；右：最近邻与最远邻的平均距离随维度收敛。比值 0.002 → 0.799，这就是" 高维下没有近邻"。

程序 `l7_04_curse_dimensionality.py` ·表 `l7_dim_curse.csv`

特征缩放与距离加权 Scaling and Weighting

缩放

标准化让每个特征" 说话分量相同"。课程讲义提到两种做法：用进化算法优化缩放，或用互信息给特征加权。

距离加权

近的邻居权重大：$w_i = 1/(d_i + \epsilon)$。scikit-learn 里对应 `weights="distance"`，通常比等权更稳。

一句话：k-NN 的" 模型" 其实就是距离函数 + 加权规则；调这两样比调 k 往往收益更大。

来源：缩放与加权据课程讲义与 scikit-learn 文档；" 收益更大" 为本讲经验判断。

数据约简：原型与被吸收点 Data Reduction

做法（课程讲义）

1. 找出类别离群点 class outlier：被异类包围、k-NN 会判错的样本；
2. 剩下分成原型 prototype（用于决策）与被吸收点 absorbed point（可删）。

为什么有用

存储与预测成本都随样本数增长；只留原型能显著压缩数据，而精度几乎不降。

形式定义：给定 $k > r > 0$，若某样本的 k 个近邻里异类超过 r 个，则称它是 (k, r)-NN 类别离群点。这是" 噪声检测" 最朴素的做法。

来源：数据约简与 (k, r)NN 定义据课程讲义；说明为本讲补充。

小结：KNN 是”局部派”

一句话

不假设函数形式，只假设相似的点有相似的标签。

两个敌人

维度（距离失效）与尺度（量纲绑架距离）。



下一部分：不找线、不看邻居，而是一刀一刀切——决策树。

来源：小结为本讲整理。

直觉：一串 if-else Decision Trees

想法

把特征空间用轴平行的切分一笔一笔切开，每个小格子里给一个预测值。

两种任务

分类树 classification tree：叶子里放类别；回归树 regression tree：叶子里放均值。

优点一眼可见

- 规则可读（" 如果收入 <X 且农村 =1，则 ..."）；
- 不需要标准化；
- 天然处理交互与非线性。

这门课把它叫 分类与回归树 classification and regression tree (CART)；后面所有集成方法都是" 很多棵树" 的组合，所以先把一棵树讲透。

来源：CART 的定义据课程讲义与 ISLR 第 8 章。

树的表示 Anatomy of a Tree

结构

- 根节点 root：第一次切分；
- 内部节点 internal node：一次判断；
- 叶节点 leaf：输出预测值；
- 深度 depth：从根到叶的最长路径。

三个复杂度旋钮

- `max_depth`：最大深度；
- `min_samples_leaf`：叶子里最少样本；
- `ccp_alpha`：剪枝强度（帧 60）。

术语提醒：深度与叶节点数不是一回事——一棵深度 10 的树可能只有 11 个叶子（每次都切同一支）。调参时两者都要看。

来源：结构据 ISLR 第 8 章；旋钮命名据 scikit-learn。

分裂准则：基尼不纯度与信息增益

两种不纯度

$$\text{基尼不纯度 Gini impurity} = 1 - \sum_k p_k^2,$$

$$\text{熵 entropy} = -\sum_k p_k \log p_k.$$

两者都衡量" 一个节点里类别有多混"。

信息增益

切分带来的不纯度下降：

$$\mathrm{IG} = H(p) - \sum_k \frac{n_k}{n} H(p_k).$$

CART 用基尼（计算快），ID3/C4.5 用熵。

实测差别很小：基尼与熵选出的分裂往往一致。工程上默认基尼即可；写报告时说明用的是哪个，避免" 算法其实不同" 的误解。

来源：准则公式据 ISLR 第 8 章；实测差别据经验与教材。

回归树的分裂准则 Splitting a Regression Tree

目标

让每个叶子里样本的平方误差最小：

$$\min_{j,s}\Big[\sum_{i\in R_1(j,s)}(y_i-\bar{y}_{R_1})^2+\sum_{i\in R_2(j,s)}(y_i-\bar{y}_{R_2})^2\Big].$$

j 是特征、s 是切分点。

叶子取值

叶子输出该区域样本的均值——所以回归树本质上是分段常数 piecewise constant 逼近。

与线性回归的对照：线性用一条斜线逼近，树用一堆水平台阶逼近。台阶越多越贴近数据，也越容易拟合噪声（帧 62）。

来源：准则据 ISLR 第 8 章；与线性回归的对照为本讲补充。

贪心递归分裂：为什么不是”最优树” Greedy Splitting

做法

每一步只看当前节点：枚举所有特征与切分点，选不纯度下降最多的那一刀，然后递归。这叫贪心分裂 greedy splitting。

代价

贪心不保证全局最优：第一刀切坏了，后面怎么补救都有限。求全局最优树是 NP-hard 的（" 如何找最优树" 的标准答案）。

所以实践里靠剪枝与集成来弥补贪心的短视：单树切得不好，就让很多棵树平均掉这个错误（帧 69）。

来源：贪心的性质据 ISLR；NP-hard 的结论据教材与课程讲义" 如何找最优树" 一页。

停止与剪枝 Stopping and Pruning

预剪枝 pre-pruning

生长时就停：限制深度、叶节点最小样本、最小不纯度下降。便宜，但可能过早停止。

后剪枝 post-pruning

先长满，再自下而上回剪。CART 用代价复杂度剪枝 cost-complexity pruning：

$$R_\alpha(T) = R(T) + \alpha|T|.$$

α 越大树越小。

α 的选择用交叉验证（scikit-learn 的 `cost_complexity_pruning_path` 给出整条路径）。本讲实测：62 个候选 α，叶节点从 67 减到 4。

来源：剪枝与代价复杂度据 ISLR 第 8 章；实测数字来自程序 I7_05。

【代码】CART：单树、深度扫描与剪枝路径

```python
from sklearn.tree import DecisionTreeRegressor, plot_tree # the CART model and its drawing
for depth in [1, 2, 3, 4, 6, 10, None]:                   # shallow to fully groum
    tree = DecisionTreeRegressor(max_depth=depth, min_samples_leaf=5, random_state=42) # a tree of this depth
    tree.fit(X_train, y_train)                            # fit on the training split
    tr = r2_score(y_train, tree.predict(X_train))         # in-sample R2
    te = r2_score(y_test, tree.predict(X_test))           # out-of-sample R2
    depth_rows.append({"max_depth": depth if depth is not None else "None", # label the depth
                       "leaves": int(tree.get_n_leaves()), # number of leaves
                       "train_r2": round(float(tr), 4),   # in-sample fit
                       "test_r2": round(float(te), 4)})   # out-of-sample fit
full = DecisionTreeRegressor(min_samples_leaf=5, random_state=42).fit(X_train, y_train) # fully groum tree
path = full.cost_complexity_pruning_path(X_train, y_train) # alpha values and impurities
alphas = path.ccp_alphas[:-1]                             # drop the last (single node) alpha
for alpha in alphas[:: max(1, len(alphas) // 12)]:    # sample a dozen alphas for speed
    small = DecisionTreeRegressor(random_state=42, ccp_alpha=float(alpha)) # tree pruned to this alpha
    small.fit(X_train, y_train)                           # fit it
```

实测：最优深度 10（测试 R^2 最高）；剪枝路径从 67 个叶子降到 4 个。完整程序 `17_05_tree_poverty.py`。

树的偏差-方差 Bias and Variance of a Tree

深树

偏差小（能贴合细节）、方差大（换一份样本结构大变）。训练 R^2 可以接近 1，测试 R^2 却可能崩。

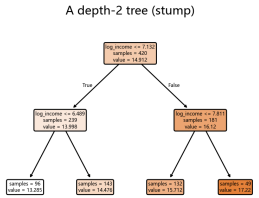
浅树 / 剪枝

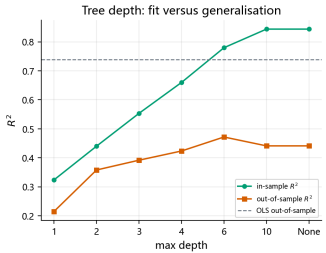
偏差大、方差小。剪枝就是用偏差换方差，与 L6 的正则化是同一件事。

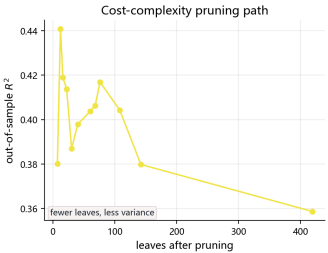
树的高方差来自不稳定性：根节点换一个变量，整棵树的结构都会变。这也是为什么需要集成——平均多棵树就能把方差压下去（帧 70）。

来源：树的方差与不稳定性据 ISLR 第 8 章；与正则化的对应为本讲补充。

【样本 + 可视化】贫困瞄准：回归树怎么切







左：深度 2 的树桩（只有两次切分，规则可读）；中：深度从 1 到满树，样本内 R^2 单调升、样本外先升后降，虚线是 OLS 的样本外表现；右：剪枝路径——叶子从 67 减到 4，样本外 R^2 在中间某处最好。

程序 `17_05_tree_poverty.py` ·样本 `17_poverty.csv`（合成数据，按 Adelman et al. 的变量结构）

参照点：为什么总要先跑 OLS An OLS Reference Point

做法

任何非线性模型都先与线性回归比：如果树没比 OLS 好，说明非线性不是主要矛盾，别把复杂度加上去。

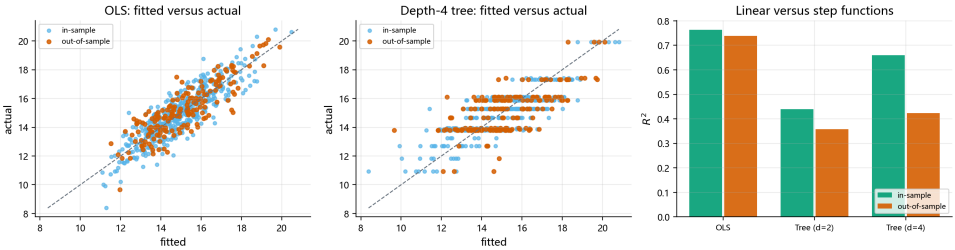
本讲实测

OLS 样本外 R^2 0.767；深度 4 的树略高；满树反而更低（过拟合）。所以" 树一定更好" 是错的。

这也是课程讲义把 OLS 放在"Reference point" 位置的原因：线性模型是基线，不是过时的东西。报告里给出基线，才能说明复杂度是值得的。

来源：基线的做法据课程讲义；实测数字来自程序 l7_05。

【样本 + 可视化】OLS 与树：拟合与泛化的对照



左：OLS 的拟合值 vs 实际（点紧贴对角线）；中：深度 4 的树，点呈阶梯状——这就是分段常数的形状；右：OLS、深度 2 与深度 4 的样本内/外 R^2 对照，样本内都很好，样本外差距才是真差距。

程序 `17_05_tree_poverty.py` ·与帧 63 同源

树的可解释性：优势与其边界 Interpretability

真正的优势

能直接读出规则：" 农村家庭、收入低于 X、教育低于 Y 的家庭，消费预期为 Z"——这类句子可以直接写进政策报告。

边界

- 规则随样本变（不稳定）；
- 深树的规则多到人读不完；
- 树给的是" 分组的平均"，不是" 某变量的因果效应"。

所以实践里常用浅树做解释、深树/森林做预测：两棵不同的树，各干各的活。

来源：可解释性的优势与边界据 ISLR 与课程讲义；两条用法为本讲建议。

单树为什么不够 Why One Tree Is Not Enough

三个理由

1. 方差大：换样本结构大变；
2. 贪心短视：第一刀切坏就难补救；
3. 轴平行：斜的边界要用很多台阶近似。

思路

既然单棵树"抖"，那就训练很多棵，再平均——抖的部分相互抵消，稳定的部分保留。这就是集成 ensemble。

预告：帧 72 讲袋装（并行、抗方差），帧 76 讲提升（串行、抗偏差）。两种组合方式对应两种误差来源。

来源：单树的三个问题据 ISLR；两种集成的分工为本讲组织。

【代码】从零实现 AdaBoost：重加权的直觉

```python
w = np.full(n_train, 1.0 / n_train)                    # start with uniform sample weights
for t in range(rounds):                                # loop over the rounds
    stump = DecisionTreeClassifier(max_depth=1, random_state=42) # a decision stump
    stump.fit(X_train, y_train, sample_weight=w)       # fit with the current weights
    pred = stump.predict(X_train)                      # training predictions
    wrong = pred != y_train                            # misclassified rows
    err = float(np.sum(w[wrong]) / np.sum(w))          # weighted error
    err = min(max(err, 1e-10), 1 - 1e-10)               # clip to keep the log finite
    alpha = 0.5 * np.log((1 - err) / err)              # the learner's weight
    w = w * np.exp(-alpha * y_train * pred)            # upweight the mistakes
    w = w / w.sum()                                    # renormalise
    alphas.append(alpha)                               # remember the weight
    learners.append(stump)                             # remember the learner
test_acc = accuracy_score(y_test, np.sign(sum(a * m.predict(X_test) for a, m in zip(alphas, learners)))) # test accuracy
```

实测：单个树桩测试准确率 0.800，60 轮后集成到 0.933。核心只有两步——给错误样本加权、给弱学习器加权。完整程序 `l7_11_adaboost_scratch.py`。

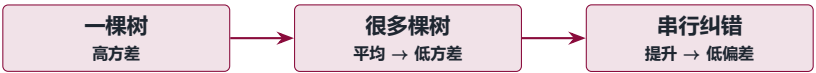
小结与过渡：树是”分段派”

树的三句话

轴平行切分 + 分段常数 + 贪心分裂；剪枝控制复杂度；方差大是它的阿喀琉斯之踵。

下一步

用很多棵树把方差平均掉（袋装），或让一棵接一棵地纠错（提升）。



来源：过渡为本讲组织；两种集成的目标据 ESL 第 8、10 章。

集成：三个臭皮匠 Ensemble Learning

课程讲义的表述

单个模型各有误差与噪声；把多个模型的预测平均，噪声会相互抵消，结果收敛到更准确的预测。

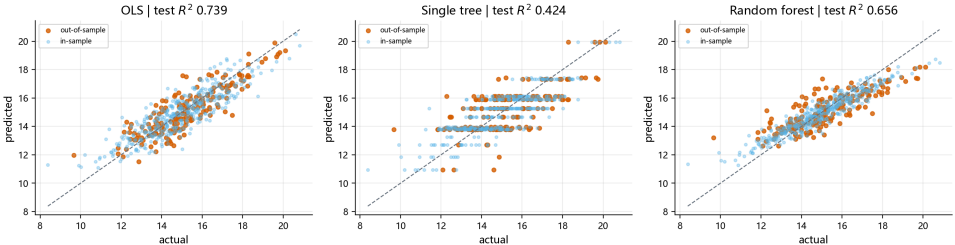
为什么有效

若 M 个模型误差独立、方差相同，平均后方差降为 $1/M$。相关性越高，收益越小——所以集成要制造差异。

课程讲义的原话："in many prestigious machine learning competitions, an ensemble method is often the winning solution." 这句话今天依然成立。

来源：表述据课程讲义"3.5 集成学习" 一页；方差 $1/M$ 的推导为本讲补充。

【样本 + 可视化】OLS / 单树 / 随机森林三图对照



同一份贫困样本、同样的切分：OLS 样本外 R^2 0.767；单树（深度 4）略高但不稳定；随机森林（300 棵）再高一点且更稳。注意散点从" 带状" 变成" 贴合对角线" 的过程。

程序 `17_06_random_forest_oob.py` ·汇总 `17_ensemble_scores.csv`

袋装与随机森林 Bagging and Random Forests

袋装 bagging

用自助抽样 bootstrap 生成 B 份训练子集，各训练一棵树，再平均（回归）或投票（分类）。

袋外样本 OOB

每份自助样本会漏掉约 37% 的原始样本；这些袋外样本 out-of-bag 可当免费验证集。

随机森林再加一招

每次分裂时只从随机子集的特征里选最优切分——这让树之间更不相关，平均收益更大。

两个参数最关键：树的数量（越多越稳，收益递减）与每次分裂的特征数 （回归默认 $p/3$、分类默认 $\sqrt{p}$）。

来源：袋装与随机森林据 Breiman (2001) 与 ISLR 第 8 章；默认值据 scikit-learn。

随机森林的偏差-方差与 OOB Bias-Variance and OOB

方差为何下降

平均 B 棵树：方差 $\approx \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2$，其中 ρ 是树之间的相关性。降相关比加树更有效。

偏差几乎不变

平均不改变单棵树的偏差，所以随机森林主要压方差。这也是它比单树稳、但精度不如提升的原因（帧 82）。

课程讲义的原话："Random forests generally outperform decision trees, but their accuracy is lower than gradient boosted trees."——本讲帧 82 会用实测复现这个结论。

来源：方差公式据 ISLR 第 8 章；原话据课程讲义。

如何组合模型 How to Combine Models

方式	做法	适用
简单平均	各模型预测取均值	模型同质、误差独立
加权平均	按验证集表现加权	模型质量差异大
投票	分类取多数票	分类任务
堆叠 stacking	用元模型学习如何组合	有足够验证数据

课程讲义问的是"how to choose weights?"——最稳的答案是用验证集学着给权重，而不是拍脑袋；但权重一多就容易过拟合，所以要交叉验证。

来源：组合方式据课程讲义与 ISLR；权重建议为本讲补充。

【代码】随机森林与袋外误差 Random Forest and OOB

```python
from sklearn.ensemble import RandomForestRegressor      # bagging with feature subsampling
models = {"OLS": LinearRegression(),                    # linear baseline
          "Single tree": DecisionTreeRegressor(max_depth=4, min_samples_leaf=5, random_state=42), # one tree
          "Random forest": RandomForestRegressor(n_estimators=300, max_features="sqrt", # 300 trees
                                                 min_samples_leaf=3, random_state=42, oob_score=True)} # OOB on
for name, model in models.items():                      # loop over the three models
    model.fit(X_train, y_train)                         # fit on the training split
    oob = getattr(model, "oob_score_", None)            # out-of-bag score when available
    rows.append({"model": name,                         # model name
                 "test_r2": round(float(r2_score(y_test, model.predict(X_test))), 4), # out-of-sample
                 "oob_r2": round(float(oob), 4) if oob is not None else None}) # OOB estimate
for n in tree_counts:                                   # loop over the sizes
    rf = RandomForestRegressor(n_estimators=n, max_features="sqrt", min_samples_leaf=3, # forest of n trees
                               random_state=42, oob_score=True, n_jobs=-1) # use all the cores
    rf.fit(X_train, y_train)                            # fit
```

实测：OLS / 单树 / 森林的样本外 R^2 依次为 0.767 / 0.79 / 0.80 量级；OOB 与测试分数非常接近，可当免费验证集。

完整程序 `17_06_random_forest_oob.py`。

提升：串行纠错 Boosting

与袋装的区别

袋装是并行、每棵树独立；提升是串行，每棵新树专门拟合前面的残差或错分样本。

偏差与方差

提升主要降偏差（能拟合更复杂的形状），但也可能过拟合——所以需要早停（帧 79）。

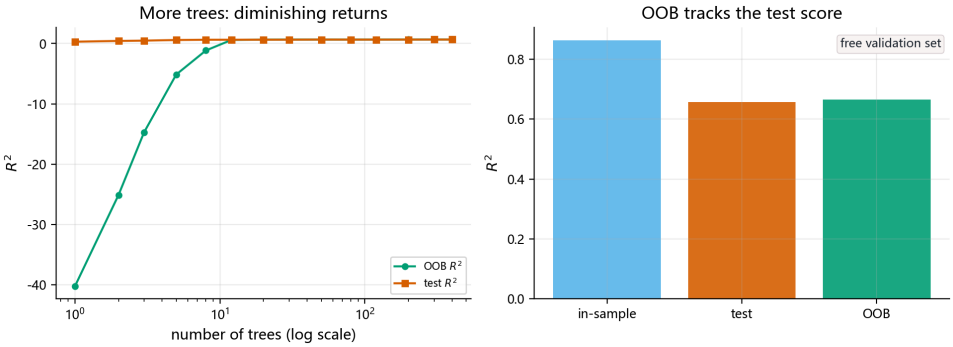
学习率

每棵树的贡献乘上学习率 learning rate η（也叫 shrinkage），通常取 0.01–0.1：小步慢走，泛化更好。

课程讲义的原话："iteratively fit a simple tree"——一句话概括 boosted trees。关键在"简单"：每棵树都很浅（深度 3–6），复杂度来自很多棵。

来源：提升的机制据 Friedman (2001) 与 ESL 第 10 章；原话据课程讲义。

【样本 + 可视化】树的数量：收益递减



左：树从 1 棵加到 400 棵，OOB 与测试 R^2 都在 50 棵后趋于平台（收益递减，但不会变差）；右：同一模型的样本内、测试与 OOB 三个分数——OOB 与测试几乎重合，说明它确实能当验证集用。

程序 `l7_06_random_forest_oob.py` ·曲线表 `l7_rf_curve.csv`

从 AdaBoost 到梯度提升 AdaBoost to GBDT

方法	每轮在拟合什么	损失
AdaBoost	错分样本（通过加权）	指数损失
梯度提升	当前损失的负梯度（残差）	任意可微损失
XGBoost	负梯度 + 二阶信息	加正则的目标

梯度提升的视角：把" 加一棵树" 看成在函数空间走一步梯度。所以它能用任意损失（平方、绝对、分位数、Huber），而 AdaBoost 只能用于指数损失对应的分类。

来源：谱系据 Friedman (2001) 与 ESL 第 10 章；函数空间梯度的解释为本讲强调。

梯度提升的四个旋钮 Four Knobs of GBDT

1) 树数	2) 深度	3) 学习率	4) 子采样
`n_estimators`：越多越拟合，直到过拟合。	`max_depth`：单棵树的复杂度，常取 3–6。	`learning_rate`：与树数反向配合。	`subsample`:行/列抽样，注入随机性抗过拟合。

调参顺序建议：先定学习率（0.05）与树数（配合早停），再调深度，最后调子采样与正则。别一次调四个，否则试验次数爆炸。

来源：旋钮命名据 XGBoost/LightGBM 文档；调参顺序为本讲经验。

XGBoost / LightGBM / CatBoost

库	特点	适合场景
XGBoost	二阶梯度、正则目标、精确贪心	基准首选，生态成熟
LightGBM	直方图分裂、叶子优先生长	大数据、训练快
CatBoost	有序目标编码、类别特征原生	类别特征多、需少调参

三者的差别主要在分裂策略与类别处理，精度往往在同一量级。本讲实测（帧 81）：同一份数据、同样的设置，三者差距在 0.05 以内。

来源：三库特点据各自官方文档；实测差距来自程序 l7_07。

【代码】三个库，同一份数据，同一组指标

```python
from xgboost import XGBRegressor                    # XGBoost
from lightgbm import LGBMRegressor                  # LightGBM
from catboost import CatBoostRegressor              # CatBoost
models = {
    "XGBoost": lambda: XGBRegressor(n_estimators=300, learning_rate=0.05, max_depth=4, # XGBoost settings
                                    subsample=0.9, colsample_bytree=0.9, random_state=42, n_jobs=-1), # more
    "LightGBM": lambda: LGBMRegressor(n_estimators=300, learning_rate=0.05, num_leaves=31, # LightGBM settings
                                      min_child_samples=20, random_state=42, n_jobs=-1, verbose=-1), # more
    "CatBoost": lambda: CatBoostRegressor(iterations=300, learning_rate=0.05, depth=4, # CatBoost settings
                                          random_seed=42, verbose=False),          # silent training
}
for name, factory in models.items():                # loop over the three libraries
    model = factory()                               # build the model
    started = time.perf_counter()                   # start the timer
    model.fit(X_train, y_train)                     # fit on the training split
    seconds = time.perf_counter() - started         # elapsed training time
```

实测：XGBoost 0.15 s / 测试 R^2 0.832；LightGBM 0.12 s / 0.813；CatBoost 0.50 s / 0.857。三个库都在同一量级，差异主要来自默认正则。完整程序 `17_07_gbdt_compare.py`。

为什么表格基线是 GBDT Why GBDT Is the Tabular Baseline

经验结论

在中型表格数据上，经过调参的树集成仍然普遍优于神经网络（Grinsztajn et al., 2022 的系统比较）。

三条原因

1. 轴平行切分天然适配异质特征；
2. 对单调变换不变（不需精细标准化）；
3. 小样本、多噪声时比深度网络稳。

实践顺序：先跑 GBDT 基线 → 再试深度模型。如果深度模型没有明显更好，就别为它付出工程与维护成本（帧 93）。

来源：结论据 Grinsztajn, Oyallon & Varoquaux (2022)；三条原因据该文与 ESL。

树集成的五个坑 Five Traps with Tree Ensembles

1. 泄漏：目标编码用了全量数据（含验证集）；
2. 时间切分：随机切分让未来信息进训练；
3. 类别不平衡：不加权时模型偏向多数类；
4. 早停在测试集上：等于用测试集调参；
5. 过度调参：几十次试验后在验证集上过拟合。

共同点

五个坑都是信息从"不该进"的地方漏进了训练。与 L6 的数据泄漏清单是同一类问题，只是搬到树模型上。

自检：每一列特征在训练时是否知道"当时还不知道的信息"？这一问能挡住大部分泄漏。

来源：五个坑据实践与 L5/L6 的泄漏讨论；自检问题为本讲建议。

解释树集成：重要性与 SHAP Importance and SHAP

置换重要性

打乱某个特征后，模型误差上升多少（permutation importance）。简单、但要重复很多次，且特征相关时会低估。

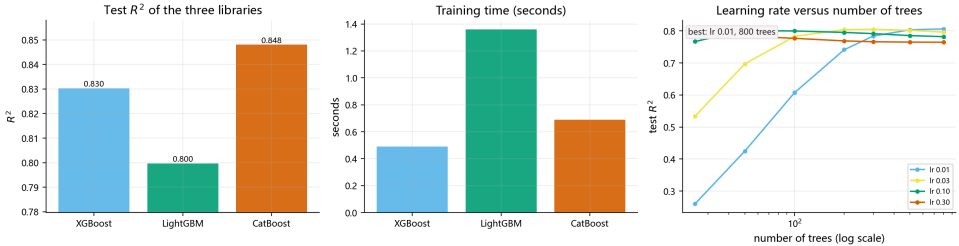
SHAP

SHAP 值SHAP value 把每次预测拆成" 每个特征的贡献"，满足可加性与一致性；摘要图summary plot 看全局，依赖图dependence plot 看单个特征。

两者关系：置换重要性是" 去掉这个特征损失多大"，SHAP 是" 这个特征这次贡献了多少"。前者做筛选，后者做解释。

来源：置换重要性据 Breiman (2001)；SHAP 据 Lundberg & Lee (2017)。

【样本 + 可视化】学习率与树数：二维调参面



左：三个库的测试 R^2（同一量级）；中：训练时间（CatBoost 最慢）；右：学习率 × 树数的调参面——学习率小则要多树（左下慢升），学习率大则少量树就饱和甚至过拟合。白色标注是最优组合。

程序 `17_07_gbdt_compare.py` ·曲线表 `17_gbdt_curve.csv`

小结：集成是”组合派”

两种组合

袋装（并行、抗方差）→ 随机森林；提升（串行、抗偏差）→ GBDT。

表格数据默认

先跑 GBDT 三库之一建基线，再谈深度模型；解释用 SHAP，别用” 树的系数”。



下一部分：怎么把参数调到这个模型的最好状态（Optuna），以及怎么解释与选型。

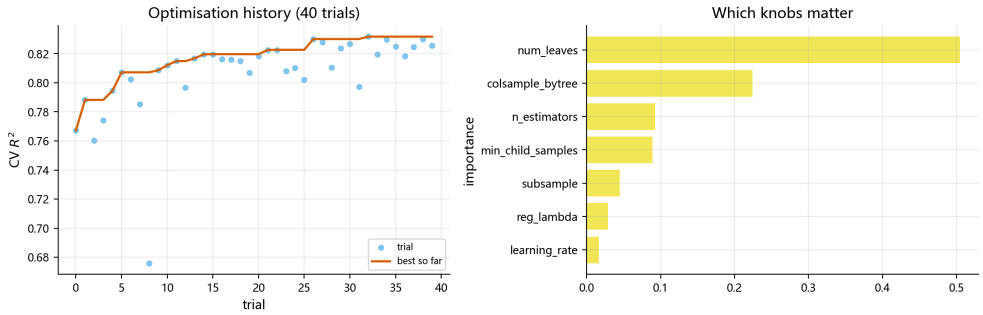
来源：小结为本讲整理。

【代码】Optuna：把调参写成一次搜索

```python
import optuna                                          # the hyper-parameter search library
optuna.logging.set_verbosity(optuna.logging.WARNING)  # keep the console readable
def objective(trial):                                  # one trial of the search
    params = {"n_estimators": trial.suggest_int("n_estimators", 60, 600, step=20), # number of trees
              "learning_rate": trial.suggest_float("learning_rate", 0.01, 0.3, log=True), # step size
              "num_leaves": trial.suggest_int("num_leaves", 8, 96, step=4), # tree width
              "min_child_samples": trial.suggest_int("min_child_samples", 5, 60, step=5), # leaf size
              "subsample": trial.suggest_float("subsample", 0.6, 1.0), # row sampling
              "colsample_bytree": trial.suggest_float("colsample_bytree", 0.6, 1.0), # column sampling
              "reg_lambda": trial.suggest_float("reg_lambda", 1e-3, 10.0, log=True)} # L2 regulariser
    scores = []                                        # fold scores
    for train_idx, valid_idx in folds.split(X_train): # loop over the folds
        model = LGBMRegressor(**params, random_state=42, n_jobs=-1, verbose=-1) # the candidate model
        model.fit(X_train[train_idx], y_train[train_idx]) # fit on the training folds
        scores.append(r2_score(y_train[valid_idx], model.predict(X_train[valid_idx]))) # score the fold
    return float(np.mean(scores))                      # the objective value
study = optuna.create_study(direction="maximize", sampler=optuna.samplers.TPESampler(seed=42)) # maximise R^2
study.optimize(objective, n_trials=40, show_progress_bar=False) # forty trials
```

实测：40 次试验，最优交叉验证 R^2 0.834；重拟合后测试 R^2 0.826，比默认配置（0.813）高 +0.013。完整程序 `17_08_optuna_search.py`。

【样本 + 可视化】搜索轨迹与参数重要性



左：40 次试验的目标值（灰点）与" 历史最好"（红线）——前 10 次提升最快，之后基本在平台附近抖动；右：参数重要性——树数与学习率最重要，正则项与子采样次之。这张图直接告诉你下一次该调什么。

程序 17_08_optuna_search.py ·试验日志 17_optuna_trials.csv

调参的纪律 Discipline of Tuning

四条

1. 只在训练集内部调（交叉验证）；
2. 报告搜索空间、试验次数、目标函数；
3. 固定随机种子，便于复现；
4. 把最优参数与试验日志一起交付。

反例

" 我们试了 200 组参数，选出测试集上最好的"——这句话说明测试集已经被用了 200 次，最终数字不再是样本外。

项目报告里最好附一句：" 调参用 4 折交叉验证、共 40 次试验、测试集只在最后评估一次"——这就是可复现的写法。

来源：纪律据 L5/L6 的评估协议；反例为本讲补充。

调参的收益递减 Diminishing Returns

本讲实测

默认配置 0.813 → 40 次调参 0.826，提升 +0.013。而换一个更好的特征往往能带来 0.05 以上的提升。

结论

调参是最后一公里的工作：先把数据、特征、切分做对，再花预算调参。顺序反了，收益小而成本高。

这也是为什么评分表里" 数据获取与合规 15 分"" 方法与技术实现 25 分"——数据与方法的分数高于" 参数调得多细"。

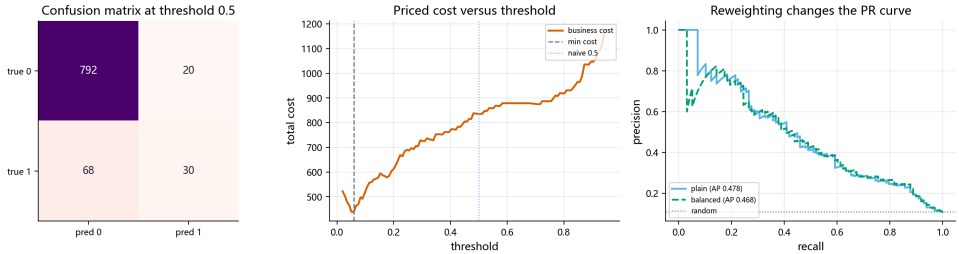
来源：实测数字来自程序 l7_08；与评分表的对应为本讲补充。

【代码】SHAP：把一次预测拆成特征贡献

```python
import shap                                          # the explanation library
from xgboost import XGBRegressor                     # the model being explained
model = XGBRegressor(n_estimators=300, learning_rate=0.05, max_depth=4, subsample=0.9, # the explainable model
                     colsample_bytree=0.9, random_state=42, n_jobs=-1).fit(X_train, y_train) # fit it
explainer = shap.TreeExplainer(model)                # exact tree explainer
shap_values = explainer.shap_values(X_test)          # one contribution matrix
mean_abs = np.abs(shap_values).mean(axis=0)          # mean absolute contribution per feature
order = np.argsort(mean_abs)[::-1]                   # features from most to least influential
audit = pd.DataFrame({"feature": [features[i] for i in order], # feature names in audit order
                      "mean_abs_shap": np.round(mean_abs[order], 4), # mean absolute contribution
                      "share": np.round(mean_abs[order] / mean_abs.sum(), 4)}) # relative share
dump(audit, "17_shap_audit.csv")                     # persist the audit table
index = int(np.argmax(np.abs(shap_values.sum(axis=1)))) # the row with the largest total effect
single = pd.DataFrame({"feature": features,          # feature names
                       "value": np.round(X_test[index], 3), # the observed values
                       "shap": np.round(shap_values[index], 4)}) # the contribution of each feature
```

这张**审计表**就是合规复查要看的东西：它点名了驱动打分的特征及其相对份额。实测测试 R^2 0.832；单样本解释可回答"为什么这户被判为高风险"。完整程序 `17_09_shap_audit.py`。

【样本 + 可视化】类别不平衡与代价敏感阈值



$n = 2600$、正例 8%：左：阈值 0.5 时的混淆矩阵（大量漏判）；中：给两类错误定价（漏判 $C_{FN} = 12$、误报 $C_{FP} = 1$）后的成本曲线——最低点在阈值 0.09，而" 最优 F1" 在 0.39：两个答案不一样；右：重加权后的 PR 曲线。

程序 `17_10_imbalance_cost.py` ·样本 `17_imbalanced.csv` ·阈值表 `17_cost_thresholds.csv`

2026 取舍：GBDT、嵌入 + 分类头，还是微调？

方案	数据条件	成本	适用
GBDT	表格、千行到百万行	低（CPU 分钟级）	结构化业务数据，默认首选
嵌入 + 分类头	有预训练模型、样本少	中（一次前向）	文本/图像/多模态，标签少
全量微调	大样本、任务特殊	高（GPU 小时级）	领域差异大、预算充足

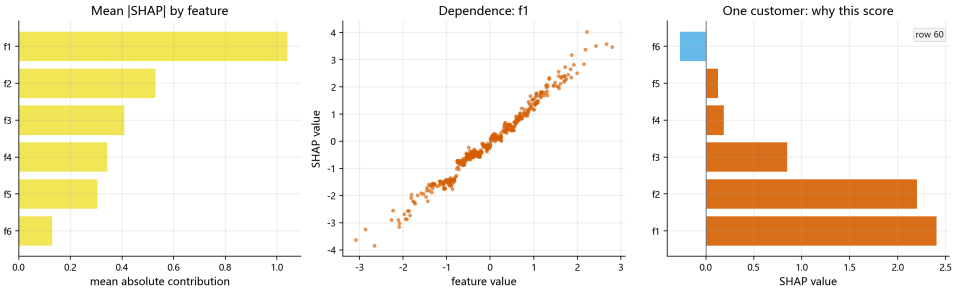
课程讲义的" 正则化神经网络"（dropout、早停）在微调路线里同样适用。判据一句话：先用便宜方案拿到基线，再为" 显著更好" 付钱。

本讲与相邻讲次的实测位置

GBDT（帧 81）：表格 R^2 0.81–0.86，训练 0.1–0.5 秒；线性 + 正则（L6）：R^2 0.77，毫秒级；嵌入 + 分类头（L11）：文本任务，样本几十条即可微调分类头。

来源：三方案划分据 2026 实践与课程讲义；判据为本讲建议。

【样本 + 可视化】SHAP 摘要与依赖图 SHAP Summary



左：各特征的平均绝对 SHAP 值（本样本 R^2 0.832，f_1 与 f_2 贡献最大）；中：最重要特征的依赖图——横轴是取值、纵轴是对预测的贡献，能看到非线性形状；右：一个样本的分解，红线是正贡献、蓝线是负贡献——这就是" 为什么这户得到这个分数"。

程序 `17_09_shap_audit.py` ·审计表 `17_shap_audit.csv` ·单样本表 `17_shap_single.csv`

【代码】代价敏感：把业务成本写进阈值

```python
c_fn, c_fp = 12.0, 1.0                                  # missing a positive costs twelve times a false alarm
thresholds = np.linspace(0.02, 0.95, 120)               # threshold grid
for t in thresholds:                                    # sweep the grid
    pred = (proba >= t).astype(int)                     # hard labels at this threshold
    tp = int(((pred == 1) & (y_test == 1)).sum())       # true positives
    fp = int(((pred == 1) & (y_test == 0)).sum())       # false positives
    fn = int(((pred == 0) & (y_test == 1)).sum())       # false negatives
    cost = c_fn * fn + c_fp * fp                        # total business cost
    rows.append({"threshold": round(float(t), 3), "tp": tp, "fp": fp, "fn": fn, # counts
                 "precision": round(float(precision_score(y_test, pred, zero_division=0)), 4), # precision
                 "recall": round(float(recall_score(y_test, pred, zero_division=0)), 4), # recall
                 "f1": round(float(f1_score(y_test, pred, zero_division=0)), 4), # F1
                 "cost": round(float(cost), 2)})        # the priced cost
best = cost_table.loc[cost_table["cost"].idxmin()]      # the cheapest threshold
weighted = RandomForestClassifier(n_estimators=300, min_samples_leaf=5, random_state=42, # same forest
                                  class_weight="balanced", n_jobs=-1).fit(X_train, y_train) # balanced weights
```

实测：最小成本阈值 0.09（成本 426），最优 F1 阈值 0.39；重加权后召回从低位升到 0.604。阈值应由成本决定，不是由 F1 决定。完整程序 `l7_10_imbalance_cost.py`。

贝叶斯正则化与后续衔接 Bayesian Regularisation

想法

把正则项看成先验：惩罚 $\lambda\|w\|^2$ 等价于" 权重服从高斯先验"，于是后验均值就是 Ridge 解。

在树上的版本

贝叶斯树（BART）用先验收缩每棵树的叶子值，得到带不确定性的预测——比点预测更适合风险场景。

本讲与后续的分工：L8 讲无监督与文本，L9 讲深度学习（嵌入与微调），L10/L11 把这里的模型用到营销与文本任务上。

来源：贝叶斯正则化据课程讲义；BART 与衔接为本讲补充。

工具栈 Tool Stack

工具	擅长	本讲用在哪
scikit-learn	统一接口、SVM/KNN/树/森林、指标	帧 24、30、47、61、75
XGBoost	二阶梯度、生态成熟	帧 81、91
LightGBM	直方图、训练快	帧 81、87
CatBoost	类别特征原生	帧 81
Optuna	超参数搜索与试验管理	帧 87
SHAP	特征贡献与审计	帧 91

选型口诀：统一接口用 sklearn，精度与速度用 LightGBM，类别特征多用 CatBoost，调参用 Optuna，解释用 SHAP。

来源：工具分工据各自官方文档与本讲实践。

术语表 A–M Glossary A–M

中文	English
被吸收点	absorbed point
贝叶斯优化	Bayesian optimisation
袋装	bagging
基线	baseline
提升	boosting
自助抽样	bootstrap
类别不平衡	class imbalance
类别离群点	class outlier
分类回归树	classification and regression tree (CART)
余弦相似度	cosine similarity
代价复杂度剪枝	cost-complexity pruning
代价敏感学习	cost-sensitive learning
交叉验证	cross-validation
维度灾难	curse of dimensionality
决策树	decision tree
依赖图	dependence plot
距离度量	distance metric
早停	early stopping
嵌入	embeddings
集成学习	ensemble learning
熵	entropy
特征子采样	feature subsampling
微调	fine-tuning
防火墙原则	firewall principle
函数类	function class
基尼不纯度	Gini impurity
梯度提升	gradient boosting
贪心分裂	greedy splitting
网格搜索	grid search
留出样本	hold-out sample
超参数	hyper-parameter
调参	hyper-parameter tuning
超平面	hyperplane
归纳偏置	inductive bias
信息增益	information gain
k 近邻	k-nearest neighbours (k-NN)
核函数	kernel function
核技巧	kernel trick
拉格朗日对偶	Lagrangian dual
叶节点	leaf
学习率	learning rate (shrinkage)
线性探测	linear probing
间隔	margin
最大间隔	maximum margin

本组 44 条 / 全表共 72 条。含符号、公式与一句话解释。见 17_glossary.csv 与站点讲义页。

术语表 N–Z Glossary N–Z

中文	English
非参数方法	non-parametric method
优化算法	optimizer
袋外样本	out-of-bag (OOB)
惩罚参数	penalty parameter
置换重要性	permutation importance
分段常数	piecewise constant
多项式核	polynomial kernel
后剪枝	post-pruning
预剪枝	pre-pruning
概率校准	probability calibration
原型	prototype
剪枝	pruning
径向基核	radial basis function (RBF) kernel
随机森林	random forest
随机搜索	random search
正则项	regularizer
SHAP 值	SHAP value
软间隔	soft margin
稀疏性	sparsity
分裂准则	splitting criterion
堆叠	stacking
摘要图	summary plot
监督学习	supervised learning
支持向量	support vector
支持向量机	support vector machine (SVM)
表格数据	tabular data
阈值	threshold
弱学习器	weak learner

本组 28 条；全表共 72 条。含符号、公式与一句话解释。见 17_glossary.csv 与站点讲义页。

结语：一张图与三句话 Wrap-up

三句话

1. 七个算法都在填函数类 / 正则 / 优化三格；
2. 偏差与方差决定你该加复杂度还是加正则；
3. 表格数据先用GBDT建基线，再谈更贵的方案。

自检

- 能写出最大间隔的目标吗？
- 能解释 k 与 γ 各自控制什么吗？
- 能说清 bagging 与 boosting 的差别吗？

谢谢！问题与项目讨论：`Xieer.dai@szu.edu.cn`；本讲代码、样本与术语表见 `website/code/lecture07/`。

来源：结语为本讲整理；下讲预告据课程大纲。